lecture_examples_quiz1

September 9, 2026

1 Lecture Examples

2 Lecture 1

Python is can be used as a calculator - operators: + , - , *, /, **, %, // - uses order of operations - integer, float, string, - built in functions: abs(), sum(), pow(), round()

3 Example 1.1

[28]:
```
# Example 1.1: Swap values of a and b

a = 2
b = 3

# You can use a built in method
a, b = b, a

print(a, b)
```

```
3 2
```

[29]:
```
# But most likely, we would declare a dummy variable c

a = 2
b = 3

c = a
a = b
b = c

print(a, b)
```

```
3 2
```

4 Example 1.2

[30]:
```
# Iterating through something
a = 1

a = 2*a + 1
a = 2*a + 1
a = 2*a + 1

print(a)
```

```
15
```

5 Lecture 2

Lists are data structures that are mutable. - left to right order - index of an item starts at 0 - you can access items by doing this: my_list[0] - you can slice a list: sliced = my_list[1:2]. This means from the second item and stop before the third index - some methods: insert(), append(), len(), max(), min(), sum()

Tuples are like lists but are immutable - used for vectors where dimension does not change

range(): generate immutable sequence of numbers - end is exclusive - range(4, -5, -3): starts at 4, ends at -4, -3 is the incremebnt - dist()

Dictionaries have keys and items

for loops - key words: for, in:, indentation - has loop body and loop variable

6 Example 2.1

[31]:
```
# Example, find 4 times this sum
# we need to find half of 2027, which is 2013.5. We should use 1015 because we␣
 ↪do not include the last digit
# or we can use incrementer and go to 2028 - 1
total = 0
for i in range(1, 1015):
    total += (-1)**(i+1)* (1 / (2*i-1))

print(4 * total)
```

```
3.1406064605356954
```

[32]:
```
# or you can just straight up use the other range
total = 0
for i in range(1, 2028, 2):
    total += (-1)**((i-1)/2)*(1 / i)

print(4 * total)
```

```
3.1406064605356954
```

7 Example 2.2

[33]:
```
# Example 2.2: Nested for loop to make a matrix
A = []
for i in range(0, 4):
    # we need a little a to be a row
    a = []
    for j in range(0, 4):
        # we need to put the stuff into a
        a.append((i+1)**j)
    A.append(a)

print(A)

# For this we need to remember that each row is gonna be created in the first␣
 ↪loop. The second (inner loop) will have each row have the actual elements
# In this case, we defined each element to be (i + 1)**j
```

```
[[1, 1, 1, 1], [1, 2, 4, 8], [1, 3, 9, 27], [1, 4, 16, 64]]
```

8 Example 2.3

[34]:
```
# Example 2.3: iteration, recurrence
# Let an+1 = an+n, where a1 = 0

a = 0
for i in range(1, 100):
    a = a + i

print(a)

# You ahve to be careful here: remember, a = 0 is already the first term. we␣
 ↪must skip that in the loop.
# Thus we need to start at 1
```

```
4950
```

9 Lecture 3

if statements will execute tasks given a condition

while loops go on for an indefinite amount of time

break and continue. break breaks out of the loop, continue skips a value in the loop

10 Example 3.1

[ ]:
```
# Example 3.1: input age of park visitor

age = int(input("Enter visitor's age: "))

print(f"The age you entered is: {age}")

if (age <= 3):
    cost = 0
elif(age < 18):
    cost = 10
elif(age < 65):
    cost = 15
else:
    cost = 5

print(f"The cost of this visitor is: ${cost}")

# Remember in the if and elif branch type stuff, if the if statement doesnt␣
 ↪execute, then it will go down the logic before it executes
```

```
The age you entered is: 66
The cost of this visitor is: $5
```

11 Example 3.2

[ ]:
```
# Example 3.2: find the smallest integer of n such that n^2 is greater than 2027

# We need a while loop bc we don't know how long it will take to find such an n

n = 1

while (n**2 < 2027):
    n += 1

print(f"The n squared greater than 2027 is {n} and the value of n^2 is {n**2}")
# if you wanted to find the largest n for which the square is less than 2027,␣
 ↪then you could just subtract 1
```

```
The n squared greater than 2027 is 46 and the value of n^2 is 2116
```

12 Example 3.3

[56]:
```
# Example 3.3: use break in a for loop to see if a list of numbers contains 0.

my_list = [2,5,1,3,56,6,2,54,0,54]

print(f"Length of list is {len(my_list)}")

for i in range(len(my_list)):
    if (my_list[i] == 0):
        print(f"There is a zero in this list at index {i}")
        break
```

```
Length of list is 10
There is a zero in this list at index 8
```

13 Example 3.4

[58]:
```
# Example 3.4: use continue to print out all the numbers less than 20 that are␣
 ↪NOT multiples of 7

my_list = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25]

print(f"Length of list is {len(my_list)}")

list_skipped = []

for i in range(len(my_list)):
    if ((my_list[i] < 20) and (my_list[i] % 7 == 0)):
        # meets the requirements, skip it
        list_skipped.append(my_list[i])
        continue
    print(my_list[i])

print(f"The list of skipped values: {list_skipped}")
```

```
Length of list is 26
1
2
3
4
5
6
8
9
10
11
```

```
12
13
15
16
17
18
19
20
21
22
23
24
25
The list of skipped values: [0, 7, 14]
```

14 Lecture 4

lambda functions are shorthand functions - used with one argument - used with multiple arguments - used with conditional logic - their syntax is: name

[8]:
```python
# lambda function examples
# Example 4.1
add_ten = lambda x:x+10
print(add_ten(5))
```

```
15
```

[10]:
```python
# Example 4.2
calc = lambda x, y: (x - y, x**y)
print(calc(3,4))

# produces a tuple
```

```
(-1, 81)
```

[12]:
```python
# Example 4.3
check = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
print(check(5))
print(check(0))
print(check(-3))
```

```
Positive
Zero
Negative
```

[13]:
```python
# Example 4.4
# Use map(). Applies a specified function to each item of an iterable (like a␣
  ↪list, tuple) and returns a lazy iterator (map object)
# Containing the transformed results
```

```
numbers = [1,2,3,4]
squared = list(map(lambda x: x**2, numbers))
print(squared)
```

```
[1, 4, 9, 16]
```

15 Lecture 5

numpy is a module. It is normally called using import numpy as np matplotlib is also a module. It is used for graphing. It is called using import matplotlib.pyplot as plt

np.linspace(0,2,6) - The last value is the number of entries. - The last value is included - Generates 6 entries from 0 to 2 - Is inclusive - Spacing is (stop - start) / (num_points - 1)

np.arange(0, 1, 0.2) - The last value is exclusive - The last argument is the increment

16 Lecture 6

[ ]:
```
# Example 6.1

# write a for loop
for i in range(1,6):
    hold = []
    for j in range(i):
        # End allows you to print on same line

        print(i, end=" ")
    print()
```

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

[ ]:
```
# Example 6.2

# Write such that we can pass c coefficients into the list

def polynomial(c, x):
    p = 0

    for i in range(0, len(c)):
        p += c[i]*x**i

    return p
```

[7]:
```
# Example 6.3
def digit_add(num):
    num = str(num)
    total = 0
    for i in num:
        total += int(i)

    return total

print(digit_add(101101))
```

```
4
```

[3]:
```
# Example 6.2

def leap_year(year):
    """
    Rules for leap years: a year is a leap year if it's divisible by 4, unless␣
 ↪it's also divisible
    by 100 but not by 400
    """

    if (year % 4 == 0):
        if (year % 100 != 0) or (year % 400 == 0):
            print(f"{year} is a leap year!")

leap_year(2024)
leap_year(400)
```

```
2024 is a leap year!
400 is a leap year!
```

[ ]:
```
# Example 6.4

def distinct_list(c):
    """
    Let c be a list with repeating numbers
    This function removes repeating numbers
    Continue skips all repeating stuff
    """
    distinct = []

    for num in c:
        if num in distinct:
            continue
        distinct.append(num)

    return distinct
```

```
old_list = [1,2,2,3,4,5,5,5]

new_list = distinct_list(old_list)
print(f"New list: {new_list}")
```

```
New list: [1, 2, 3, 4, 5]
```